An 8-bit Deep Learning Accelerator for GAN Image Generation on GF180MCU

William Anthony, I Made Medika Surya W.M., John Ruben Saragi, Mahardhika Putra Adipratama,
Fairuz Aquilla Rahagi, Trio Adiono, and Infall Syafalni
Bandung Institute of Technology, Bandung, Indonesia
E-mail: {13223048, 13222021, 13222049, 13222081, 13222107}@mahasiswa.itb.ac.id,
tadiono@itb.ac.id, infall@ieee.org

***Abstract*—This paper presents an 8-bit deep learning accelerator for a small MNIST GAN image generator, hardened through a fully open-source flow on the open GF180MCU 180 nm process. A quantized three-layer MLP generator maps onto a structural 4×4 processing-element array of native accumulation depth $K = 256$. Where a folding predecessor stored the sixteen-accumulator result buffer in a small SRAM macro, this design realizes it as a register file — cheaper and faster for a buffer the array geometry fixes at 48 bytes — removing the last non-bandwidth macro and leaving eight 256×8 SRAM macros on a rectangular 1600×1100 μm die, with single-cycle reads and 16-cycle writeback. Antenna closed on a two-lever path: the roomy floorplan left four marginal violations on long SRAM-to-array broadcast nets, which clustering the eight macros shortened at the source, and a density sweep then routed to zero ($4 \to 2 \to 0$), reproducibly. The design is verified with a Universal Verification Methodology (UVM) environment in pyuvm on cocotb — transaction-level, self-checking, with an independent reference model — and signs off with zero DRC, LVS, XOR, and antenna violations at 40 ns across nine STA corners.**

***Index Terms*—Deep learning accelerator, GF180MCU, SRAM macro, register file, antenna closure, UVM, cocotb, INT8 quantization, GAN, ASIC, LibreLane, open-source EDA.**

I. Introduction

Generative Adversarial Networks are a compact workload for testing neural inference on custom hardware. The generator used here is a small MNIST multi-layer perceptron turning a 64-dimensional latent vector into a 28×28 image; its arithmetic is dense matrix multiplication, so it maps onto a small array of multiply–accumulate units. An earlier version expressed this as behavioral loops and register memory arrays — convenient for checking the algorithm, but not implementable, since buffers inferred as standard-cell registers make the design too large to route. It was restructured into a structural accelerator with a processing-element (PE) array, SRAM-backed buffers, and a controller, and hardened with eleven 256×8 SRAM macros on a 1600×1500 μm die.

This paper takes that design's memory question one step further. A folding predecessor had already reduced the result buffer to a single 64×8 macro storing 48 bytes in 64; but a buffer the array geometry fixes at sixteen 24-bit accumulators is small enough that even the shallowest SRAM macro is over-provisioned. Realized as a register file — 384 flip-flops — it removes that macro entirely, leaving only the eight operand macros the array bandwidth demands, and returns the buffer to single-cycle reads. The contributions are: (1) a register-file result buffer that removes the last non-bandwidth macro and restores single-cycle access at negligible area; (2) an antenna-closure path for a spread-out floorplan — cluster the macros to shorten the broadcast nets at the source, then re-roll the marginal residue to zero with a density sweep — confirmed reproducible by a byte-identical re-run; (3) a Universal Verification Methodology environment, built open-source in pyuvm on cocotb, that checks the accelerator transaction-by-transaction against an independent reference model; and (4) a full physical sign-off of the eight-macro rectangular die at 40 ns across nine STA corners.

II. Related Work

Most neural accelerators are arrays of multiply-and-accumulate units — the systolic TPU [1] at large scale, surveyed broadly in [2] — and running a floating-point network on an integer array normally uses the per-tensor quantization scale of Jacob et al. [6]. The fabricated reference points span three orders of magnitude: DianNao [3], Eyeriss [4] with its row-stationary dataflow, and, closer to our scale, Wang et al. [7], all closed-PDK designs on commercial nodes. The comparison that matters most is OpenSpike [5], a spiking accelerator in open SkyWater 130 nm with open EDA and OpenRAM memory: same open-flow premise, different workload and memory strategy (Table V). The workload is a pretrained MLP GAN [8], [9]; the RTL builds on the open DLA Engine project [10].

III. GAN Architecture

The generator (Fig. 1) projects a 64-dimensional latent vector through two 256-neuron ReLU hidden layers into a 784-neuron tanh output layer, reshaped as a 28×28 grayscale image. Its dense layers are L0 (256×64, ReLU), L2 (256×256, ReLU), and L4 (784×256, tanh): 282,624 MACs. At four output neurons per tile, L0 and L2 need 64 tiles each and L4 needs 196 — 324 tiles per image.

IV. Hardware Architecture

The DLA computes a tiled matrix multiplication $C = A\times B$ with array size $N = 4$ and native accumulation depth

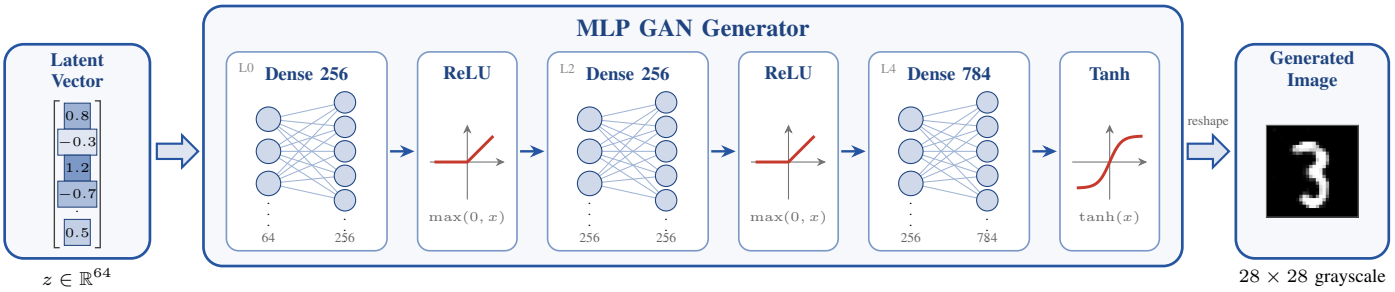

Fig. 1. MLP GAN generator: the 64-dimensional latent vector z is expanded through two 256-neuron ReLU hidden layers and a 784-neuron tanh output layer into a 28×28 grayscale image.

$K = 256$, so one transaction accumulates a full 256-term dot product — the inner dimension of the widest generator layers. K is temporal: it sets how many cycles the same 16 PEs accumulate, not the array size. An earlier RTL revision defaulted to $K = 4$ and relied on a harness override; since the physical flow synthesizes bare RTL defaults, the hardened default was widened to $K = 256$, a parameter-only change as the operand SRAMs were already 256 deep. Per transaction the controller clears the accumulators, enables the array for K cycles, and stores the result block to C through a writeback path that serializes the sixteen accumulators.

At the top level (Fig. 2) A provides one row vector to the array and B one column vector, the control unit generates the read index and the clear, enable, done, and busy signals, and the output bus carries the accumulated C values.

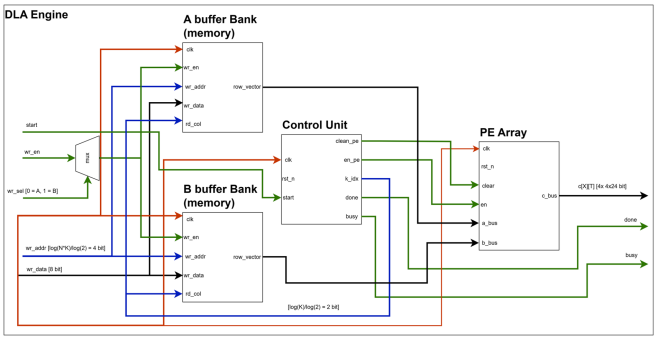

Fig. 2. DLA engine top level.

B addresses until all K terms are consumed, DONE holds the completion flag until START falls. A writeback sequencer then serializes the accumulators into C and raises a writeback-done flag.

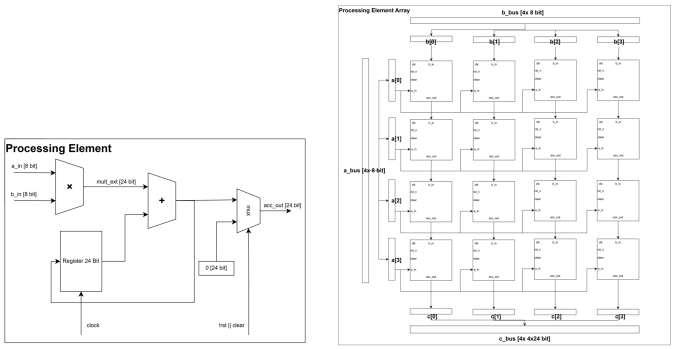

Fig. 3. Processing element datapath (left) and the 4×4 array (right).

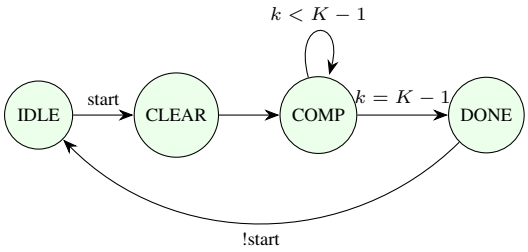

Fig. 4. Controller state machine.

A. PE Array and Controller

Each PE (Fig. 3, left) takes two signed 8-bit operands, multiplies them, sign-extends the 16-bit product to 24 bits, and adds it to a local accumulator,

$$acc[n] = acc[n-1] + a[n]\,b[n], \tag{1}$$

so after K cycles it holds one tile output; 24 bits give 256 accumulated 8-bit products enough range. The array (Fig. 3, right) is 16 PEs in a 4×4 grid, each row fed one activation lane and each column one weight lane by broadcast, each PE accumulating locally — output-stationary, all 16 in parallel, one tile per accumulation loop. The controller (Fig. 4) has four states: IDLE waits for START, CLEAR zeroes the accumulators, COMPUTE enables the array and increments the A and

B. Operand SRAMs and a Register-File Result Buffer

The operand buffers are single-port SRAM macros: four 256×8 for A, one per row lane, and four for B, one per column. Eight is a bandwidth floor, not a capacity choice — the array consumes four A and four B bytes per compute cycle from 1RW 8-bit parts, so no depth reduction reduces the count — and both banks are exactly full at 1,024 bytes. In this design these eight are the *only* macros on the die.

The result buffer is where the predecessors spent a macro and this design spends none. It holds $N \times N = 16$ accumulators of 24 bits — 48 bytes, a size the array geometry fixes forever. An eleven-macro ancestor stored it as three 256×8 byte-plane macros (203,314 μm^2); a nine-macro successor folded those onto one 64×8 macro (45,861 μm^2), at the cost of

a four-cycle read and a 48-cycle writeback. Both still paid a macro's overhead — keep-out halo, Metal3 power hookup, a fixed floorplan slot — to hold 384 bits. This design holds the 384 bits in a register file: sixteen 24-bit words of flip-flops with a registered read, addressed as $addr = i \times N + j$, which synthesis places freely in the fabric, so the ninth macro disappears. It is smaller and faster than the folded macro it replaces — roughly 400 flip-flops and a 16:1 multiplexer, on the order of 10^4 μm^2 against 45,861 — with a one-cycle write (writeback returns to 16 cycles from 48) and a next-cycle read. A buffer this small, fixed in size and off the critical path, is exactly where a register file dominates a hard macro on area, latency, and floorplan simplicity at once. The operand-SRAM wrapper is unchanged, and the one-cycle operand read is still absorbed by delaying the PE enable one cycle.

V. Chip-Level Integration

The accelerator exposes 53 signal bits, but the shuttle slot gives only 20 general-purpose digital pads beside 60 analog, 8 supply, clock, and reset pads. Rather than a pin-per-signal map, the core wraps the accelerator behind a synchronous 4-wire serial bridge (SCLK, MOSI, CS_N, MISO) that treats SCLK as data: all three inputs are double-flop synchronized and SCLK edge-detected, avoiding a second clock domain. Frames shift MSB-first while CS_N is low — 2-bit command, 10-bit address, plus 8 data bits on writes: WRITE_A and WRITE_B load the operand buffers, START launches a tile, READ_C returns a 24-bit two's-complement result on MISO, and a rising CS_N aborts safely. Three status flags (busy, done, writeback-done) drive pads directly for scope-visible liveness, so 9 of the slot's 91 pads are used. The register-file result buffer also simplifies the host protocol: READ_C returns on the cycle after its address, so the host needs none of the plane-walk delay the folded-macro ancestor imposed, whose three-plane read forced extra idle core clocks before the data.

The padring is the wafer.space GF180MCU template [11] for a 2935×2935 μm workshop slot. It shorts the pad and core supplies into one rail on every pad, so the chip is single-supply by construction, and the operating point is 3.3 V for the whole die: logic and SRAM are 3.3 V parts and the foundry I/O cells are characterized at 3.3 V, so every cell runs in a foundry-characterized corner and the I/O levels suit a 3.3 V microcontroller host directly. A predecessor completed this flow with a clean chip-level sign-off, LVS over 71,668 instances included; the padring slot is a fixed, shared frame, so **it has not been re-run against the eight-macro macro**, which is the reason this paper reports macro-level results.

VI. Software to Hardware Flow

The hardened core is the DLA engine alone; everything around it is a host (Fig. 5). In simulation that is Python plus a Verilog testbench preparing quantized weights, inputs, and reference outputs; on the finished chip, a microcontroller streaming the same tile schedule over the serial protocol.

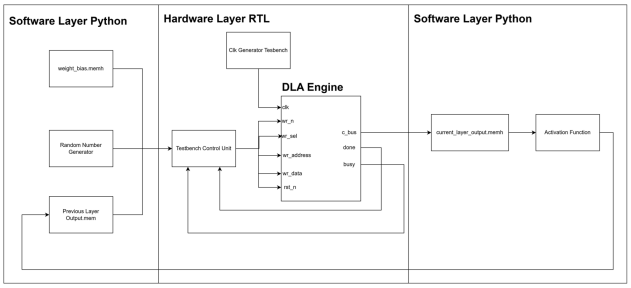

Fig. 5. Hardware and software flow.

A. Quantization and Requantization

Every weight and bias tensor becomes signed 8-bit with one scale factor per tensor, taken from its largest absolute value and recorded in a manifest the testbench reuses. The DLA returns only a tile's integer dot product; the surrounding math is integer on the host, so hardware matches the Python reference exactly. Hidden-layer results are rescaled, biased, ReLU-activated, and requantized to 8 bits; output-layer results go to fixed point, through a tanh lookup table, to a grayscale pixel.

B. Why INT8

TABLE I
Numeric formats: accuracy against an exact FP64 render (10 latents, 7,840 pixels), and hardware cost. PE area is synthesized on the 3.3 V cells; accumulator width is what 256 accumulated products need.

	Pixel error (gray levels)		Weights	PE area	Acc.
Format	mean	max	(KiB)	(μm^2)	(bits)
INT4	22.26	255	138	6,245	15
INT8	**1.22**	**72**	**276**	**15,963**	**23**
INT16	0.01	1	552	47,037	39
FP16	0.06	5	552	—	—

Operand width sets multiplier area, buffer depth, and stream volume at once, so it was chosen by measurement: each format was applied to the same FP32 checkpoint and its render compared against an exact FP64 render of the same latent, against one PE's synthesized area at that width (Table I). INT4 is not viable — a per-tensor scale leaves seven magnitude steps, and 256 accumulated terms of that coarseness destroy the digit. INT16 is essentially exact but costs $2.95\times$ the PE area, a 39-bit accumulator, and double the weight traffic, which doubles time per image since streaming dominates latency (Section IX). FP16 is no more accurate than INT16 for the same memory and a different datapath; none was built, so no area is quoted. INT8 sits at the knee: 1.2 gray levels of mean error is invisible in a 28×28 digit, the 16-bit product fits the 24-bit accumulator, and the width matches the SRAM word, so buffers need no packing logic.

VII. Verification

The generator maps onto the DLA as tiled dense layers: per tile the four weight rows go to A and the shared input vector

to B, the array performs the dot products, and the results land in C, one datapath serving every layer.

A. Golden-Vector Regression

A directed suite passes against Python golden vectors: a GEMM on a single lane; a full $N{=}4/K{=}256$ tile exercising every A and B macro; one layer row; a full 784-pixel image; and the pad-level serial protocol driven as external hardware would. Because the result buffer is now a register file, the read-timing complications the folded macro imposed on every caller — a plane-walk restart on address change, an extra wait cycle in the serial bridge — disappear: a read is simply valid on the next cycle, the same contract as the operand SRAMs.

B. UVM Verification

Beyond directed vectors, the accelerator is verified with a Universal Verification Methodology (UVM, IEEE 1800.2) environment. UVM is normally SystemVerilog on a commercial simulator; to keep the flow open-source, the environment is built in *pyuvm*, a faithful re-implementation of the UVM class library, driving the design through cocotb over Icarus Verilog, so every component has its SystemVerilog counterpart. The hierarchy is standard (Fig. 6): a *sequence* generates transactions, each a full $A \times B$ tile with random signed operands; a *sequencer* and *driver* apply them at the pins — load A and B, pulse START, read the sixteen results; a passive *monitor* reconstructs the observed result matrix from the read-back bus; and a *scoreboard* predicts $C = A \times B$ with an independent 24-bit integer reference model and compares. Everything above the driver speaks in transactions rather than pin values, decoupling stimulus and checking from the protocol. A directed all-ones tile ($C_{ij} = K = 256$) and constrained-random tiles all pass, sixteen outputs each. The value of the UVM layer over directed vectors is structural: the scoreboard's reference is independent of the stimulus, the monitor observes the true output pins as a black box, and adding coverage or new sequences needs no change to the checker — the reusability directed testbenches lack.

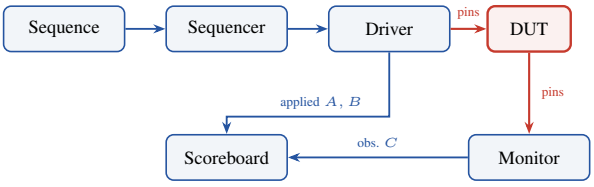

Fig. 6. UVM environment: stimulus flows as transactions from the sequence to the driver, which applies it at the pins; the monitor reconstructs the result and the scoreboard checks it against an independent reference model fed the applied operands.

C. Gate-Level and End-to-End

Gate-level re-simulation on the routed eight-macro netlist — inserted buffers and roughly 10,000 antenna diodes included — reproduces the golden results bit-exactly, closing the gap between correct RTL and a correct taped-out netlist. The rendered digits (Fig. 7) are limited by the small generator and INT8 quantization but confirm the full software-to-hardware path runs end to end.

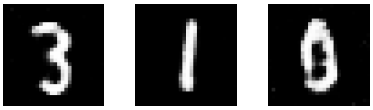

Fig. 7. Digits generated through the hardware path.

VIII. Physical Implementation

The design is implemented with LibreLane [12] on the GF180MCU open PDK [13], hardened as a macro whose GDS/LEF/liberty views a padring integration consumes; everything below is that macro.

A. Single-Supply 3.3 V Library Migration

The buffer SRAM [14] is a 3.3 V IP but the open PDK ships only 5 V cells, so a first version mixed a 5 V core with 3.3 V SRAMs — fine for layout checks, level-shifted on silicon. Migrating to the third-party `gf180mcu_as_sc_mcu7t3v3` native-3.3 V library, which ships nine liberty corners and a LibreLane integration, makes the die single-supply; bring-up surfaced three bugs in its flow integration (a missing timing-corner variable, a synthesis-exclusion filename mismatch, and non-synthesizable `$random` initializers in six sequential-cell models that crashed lint), all fixed at the library source. The 3.3 V cells are far faster than the 5 V ones: the first run at 75 ns closed with $+44$ ns of setup slack, so the clock was retimed to 40 ns (25 MHz).

B. Hardening the Accelerator Macro

TABLE II
ANTENNA CLOSURE ON THE EIGHT-MACRO DIE (40 NS, 1600 × 1100 μM). ALL RUNS ARE DRC-, LVS-, AND TIMING-CLEAN; ONLY ANTENNA VARIES. VIOLATING NETS ARE LONG SRAM-TO-ARRAY BROADCAST AND BUFFER-FANOUT NETS.

Placement	Density	Nets	Worst	Note
roomy grid	56	4	1.44×	baseline
roomy, margin 40	56	4	1.44×	diodes no help
cluster 55 μm	48	2	1.26×	broadcast shortened
cluster 55 μm	44	1	—	sweep
cluster 55 μm	**50**	**0**	—	**sign-off**
cluster 55 μm	52	2	—	sweep

Four configuration points were essential: a synthesis define keeps the SRAM a hard macro (without it the memory is silently built from flip-flops); the eight macros are placed explicitly; the SRAM power pins reach only Metal3, so the macro power connection is made there; and routing runs to Metal5, with cell padding for diode access and diodes inserted at placement, in-router repair disabled — its verification reroute aborts on clock-net diode pin access.

Antenna closure on the eight-macro floorplan took a different path from the folding predecessor's (Table II). Laid out first on a roomy 1600×1100 μm die with the macros spread evenly, the design signed off clean except for four marginal antenna violations (worst $1.44\times$), all on long SRAM-to-array

broadcast nets: with the macros far apart, a row or column broadcast crossed a large fraction of the die. Two levers were tried in isolation. Raising the pre-route diode margin left the count at four — extra diodes cannot shorten a wire, so a length-dominated violation is unmoved. Clustering the eight macros into a compact central 4×2 block (channels tightened to 55 μm) attacked the cause instead: the placer packs the datapath logic against the clustered memories, so the broadcast nets shrink, and the count fell to two. A density sweep over the clustered floorplan then re-rolled the last two marginal fanout nets: densities of 44, 48, 50 and 52 gave one, two, **zero**, and two violations, non-monotonically, and 50 reached zero. The result reproduces — an independent re-run produced a byte-identical placement-and-routing database — so it is a found configuration, not a routing-thread fluke.

The lesson generalizes: antenna on this flow is a placement problem, not a repair-tool one — shorten the offending nets, whether by moving one mis-placed macro (as the nine-macro predecessor did) or by clustering all of them, then let a density perturbation route the marginal residue clean.

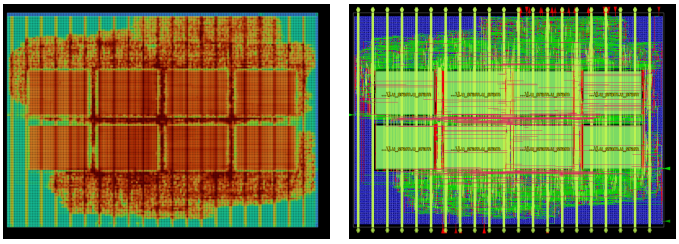

Fig. 8. Hardened macro, 1600×1100 μm: placement-density heatmap (left; red packed, blue empty) and the place-and-route view (right, macros labeled). The eight SRAM macros cluster into a central 4×2 block — four A on the lower row, four B on the upper — and the datapath logic packs against them, keeping the SRAM-to-array broadcast nets short; the result buffer is a register file, not a macro.

Fig. 8 shows the resulting layout and Table III the sign-off: zero Magic DRC errors, zero Netgen LVS errors over 23,540 devices, zero GDS XOR differences, zero violating antenna nets and pins, and the 40 ns clock met at all nine STA corners. Static IR-drop analysis reports a worst on-die supply droop of 0.80 mV on VSS and 0.35 mV on VDD against 3.3 V with zero power-grid violations — read as the on-die grid only, since a core macro carries no package or pad model. The nominal 121 mW is almost entirely internal power estimated with the flow's default switching activity, and should be read as a conservative tool estimate rather than a measurement.

C. Area and Timing Detail

The core stays memory-dominated: eight macros take 0.542 mm^2 (32% of the core), a bandwidth floor with no result-buffer macro left to remove. The register file costs $\sim 10^4$ μm^2 of standard cells against the folded macro's 45,861 and adds $\sim$400 flip-flops; instances rise 7% over the nine-macro design (register file plus the roomier die's fill and 10,002 antenna diodes) and wirelength 23%. Timing signs off at nine corners with zero violations, setup at the slow corner (SS, 125 °C, 3.0 V; +15.74 ns) and hold at the fast one (FF; +0.118 ns, the thinnest number, set by clock-tree skew). The true critical path is near 24 ns, unchanged by the C-buffer decision since it runs through the A and B macros, not the result buffer, so 25 MHz is a guard band. Table III collects the comparison: removing the result-buffer macro trims die and power and restores single-cycle reads and 16-cycle writeback — compute per image drops 10% to the eleven-macro figure — paying only in instances and wirelength.

TABLE III
SIGN-OFF OF THE EIGHT-MACRO REGISTER-BUFFERED DESIGN AGAINST THE NINE-MACRO FOLDED-BUFFER PREDECESSOR. BOTH ARE HARDENED ON GF180MCU 180 NM AT 3.3 V WITH THE GF180MCU_AS_SC_MCU7T3V3 CELLS AT 40 NS, AND BOTH CLOSE WITH 0 DRC, 0 LVS, 0 XOR, AND 0 ANTENNA VIOLATIONS. POWER IS A TOOL ESTIMATE.

	9 macros	8 macros	Change
Die (μm)	1375×1325	1600×1100	
Die area (mm^2)	1.822	1.760	−3%
Core area (mm^2)	1.761	1.692	−4%
SRAM macros	9	8	−1
C buffer	64×8 macro	register file	
C buffer area (μm^2)	45,861	∼10,000	std cells
Instances	79,676	84,875	+7%
Power (mW)	133	121	−9%
Wirelength (mm)	812	997	+23%
Writeback (cycles)	48	16	−67%
C read latency (cycles)	4	1	−75%
Compute per image (ms)	3.95	3.54	−10%
Setup slack, worst (ns)	+16.24	+15.74	
Hold slack, worst (ns)	+0.116	+0.118	

IX. PERFORMANCE

A. Latency

TABLE IV
END-TO-END LATENCY FOR ONE 28 × 28 IMAGE AT 25 MHZ AND SCLK 3.125 MHZ (CLK/8), DERIVED FROM THE PER-TILE CONTROLLER CYCLE COUNT (CLEAR + 256 ACCUMULATE + 16 WRITEBACK) AND THE PROTOCOL FRAME LENGTHS.

Stage	Frames	Time (ms)	Share
Weight tiles → A buffer	331,776	2,123.4	98.86%
Input vector → B buffer	768	4.9	0.23%
START commands	324	1.2	0.06%
Compute + writeback (on chip)	324 tiles	3.5	0.16%
Result readback from C	1,296	15.3	0.71%
Total		2,148	0.47 img/s

The array does 16 MACs per cycle (peak 0.8 GOPS at 25 MHz) but is interface-bound. On-chip compute is 273 cycles per tile — CLEAR, 256 accumulation, and a 16-cycle register-file writeback — so 324 tiles take 88,452 cycles, 3.54 ms at 40 ns, against 3.95 ms for the folded-macro predecessor whose writeback was 48. Feeding it dominates (Table IV): each tile streams four 256-deep weight rows as one 20-bit frame per byte at SCLK $\leq$ clk/8, so weight loading alone is 2.12 s and end-to-end latency is **2.15 s per image** (0.47 img/s), the link 99.8%. The bound is architectural, not port width: a matrix–vector product reuses no weight, so 1.01 bytes cross per MAC however the interface is built. Batching latents through the twelve idle PE columns, a burst write sending the

address once (removing 12 of every 20 bits), and a wider bus are the levers, none a datapath change.

B. Comparison

TABLE V
COMPARISON WITH REPRESENTATIVE ACCELERATORS. THROUGHPUT IS PEAK (2× MACS × FREQUENCY); EFFICIENCY IS PEAK THROUGHPUT OVER REPORTED POWER. OUR POWER IS A TOOL ESTIMATE.

Design	Tech. (nm)	Area (mm^2)	Peak (GOPS)	Eff. (GOPS/W)	Open flow
DianNao [3]	65	3.02	452	932	No
Eyeriss [4]	65	12.25	67.2	242	No
Wang *et al.* [7]	40	11.97	54.61	567	No
OpenSpike [5]	130	33.3	—	56.8	Yes
This work	180	**1.76**	0.8	6.6	**Yes**

Table V places this work against three fabricated commercial-node accelerators [3], [4], [7] and one open-PDK, open-flow tapeout [5], on peak figures so different scales compare on one basis. The gap reads plainly: two to three orders of magnitude below the commercial parts in throughput, about two in efficiency, almost none of it architectural. A 4×4 array at 25 MHz has $31\times$ fewer multipliers than Eyeriss at $8\times$ the clock, which alone accounts for throughput; the efficiency gap is what 180 nm at 3.3 V costs. The isolating comparison is OpenSpike: at similar scale and clock this design occupies 1.76 mm^2 against 33.3 mm^2, on foundry hard SRAM macros rather than generated memory. What this work offers is reproducibility and density, not performance.

X. CONCLUSION

This work refines a signed-off INT8 GAN-generator accelerator by removing its last non-bandwidth memory: the sixteen-accumulator result buffer, over-provisioned by even the shallowest SRAM macro, becomes a register file, leaving only the eight operand macros on a rectangular 1600×1100 μm die with single-cycle reads and 16-cycle writeback. Antenna closed by clustering the macros to shorten their broadcast nets and re-rolling the residue to zero with a density sweep, reproducibly, and the design is verified transaction-by-transaction by an open-source UVM environment against an independent reference model, signing off with zero DRC, LVS, XOR, and antenna violations at 40 ns across nine corners on one 3.3 V supply. Two lessons carry: a small, fixed, off-critical-path buffer belongs in a register file, not a hard macro; and antenna here is a placement problem — cluster to shorten the nets, then let density route the residue clean. The workload stays interface-bound at any port width, so batching, burst writes, and a wider bus are the levers. Next is the padring integration, then silicon.

ACKNOWLEDGMENT

The authors thank the open-source EDA community, the GlobalFoundries open PDK program, and the wafer.space chipathon program.

REFERENCES

[1] N. P. Jouppi *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. ISCA*, 2017.
[2] C. Silvano *et al.*, "A survey on deep learning hardware accelerators for heterogeneous HPC platforms," *ACM Computing Surveys*, 2025.
[3] T. Chen *et al.*, "DianNao: a small-footprint high-throughput accelerator for ubiquitous machine learning," *Proc. ASPLOS*, 2014.
[4] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: an energy-efficient accelerator for deep convolutional neural networks," *IEEE JSSC*, vol. 52, no. 1, pp. 127–138, 2017.
[5] F. Modaresi, M. Guthaus, and J. K. Eshraghian, "OpenSpike: an OpenRAM SNN accelerator," in *Proc. IEEE ISCAS*, 2023.
[6] B. Jacob *et al.*, "Quantization and training of neural networks for integer-arithmetic-only inference," *Proc. CVPR*, 2018.
[7] C. C. Wang *et al.*, "A 54.61 GOPS 96.35 mW digital logic accelerator for underwater object recognition DNN in 40 nm CMOS," in *Proc. IEEE AICAS*, 2024.
[8] I. Goodfellow *et al.*, "Generative adversarial nets," in *Proc. NeurIPS*, 2014.
[9] C. Singh, "GAN/VAE pretrained PyTorch models," github.com/csinva/gan-vae-pretrained-pytorch.
[10] "DLA Engine," GitHub, github.com/wlmoi/DLA_Engine.
[11] wafer.space, "GF180MCU project template," github.com/wafer-space/gf180mcu-project-template.
[12] "LibreLane: an open-source RTL-to-GDSII flow," github.com/librelane/librelane.
[13] GlobalFoundries and Google, "GF180MCU open-source PDK," github.com/google/gf180mcu-pdk.
[14] R. T. Edwards, "GF180MCU OCD IP SRAM macros," github.com/RTimothyEdwards/gf180mcu_ocd_ip_sram.